BEYOND AI WITH LAKI · DAY 05

AI’s cheat sheet for the next word

KV CACHE · COMPLETE LEARNING GUIDE

Football intuition, worked mathematics, memory planning and interview practice. Beginner to AI engineer.

1. The commentator who keeps notes

She shoots. The keeper dives. It hits the post, and it’s in! Imagine a commentator replaying the entire match before every new sentence. The celebration would be over before the commentary caught up.

Our football Reel gives the commentator a desk of notes. In a language model, the useful saved material is numerical: **keys and values from previously processed tokens**. Keeping these intermediate results is called a **key-value cache**, or KV cache.

The idea: reuse earlier computation while generating the next token. There is still work to do for each new token, and storing the saved results costs memory.

Where the football analogy stops

A real cache contains tensors, not written summaries of match events. A token may be part of a word, punctuation or a whole word. Our word cards are a simplified illustration. A model also does not watch a match or consciously decide what to remember.

Your learning route

- Start with the intuition and follow one generation step.
- Calculate a small attention example by hand.
- Estimate how many bytes a cache needs.
- Use the engineering checklist and interview questions to go deeper.

Prerequisites: a token is a unit processed by a model; a vector is a list of numbers. See Day 2 and Day 3 for background.

2. What are queries, keys and values?

Attention combines information from token positions. At a particular layer and attention head, the current token creates a **query (Q)**. It is compared with the **keys (K)** of allowed positions. The resulting scores become weights used to combine their **values (V)**.

A useful intuition, not a literal dictionary

Object	Role	Commentator analogy
Query	Vector used to score relevant positions	What information do I need now?
Key	Vector compared with the query	Clues for locating an entry
Value	Vector contributing to the weighted result	Information available from that entry

These vectors are learned projections of the layer's input representation. They are not human-readable labels. Deeper layers receive representations already transformed by earlier layers.

$$q = xW_Q$$
$$k = xW_K$$
$$v = xW_V$$

Here x is a row vector for one token at this layer. W_Q, W_K and W_V are learned matrices. This simplified notation omits biases, position operations and head reshaping. The values differ across layers and heads.

Why keep K and V rather than old Q?

In incremental causal decoding, the new query reads the existing keys and values. Earlier queries were used for earlier positions; they are not needed to evaluate this new position's attention. The reusable state is therefore K and V. Each attention layer maintains its own cache.

Further reading: [1], [2].

3. Follow a complete decoding step

Consider the illustrative prefix **“She shoots it hits”**. For this lesson only, treat each word as one token. In practice a tokenizer determines the actual pieces.

Two phases of generation — simplified causal Transformer

Prefill · Process the prompt; build layer caches	Decode · Process a new token; reuse K/V	Repeat · Choose the next token and continue

1. **Prefill:** process the prompt, usually with parallel computation across its positions under a causal mask. Store each layer’s keys and values. The final prompt position’s output predicts the first generated token.

2. **Select:** choose a next token from the output distribution, for example “the”. Selection might use greedy decoding or sampling.

3. **Process “the”:** at each layer, compute its new query, key and value. Combine its fresh key/value with the saved entries. Its query attends to the allowed past positions and its own current position.

4. **Predict again:** the final-layer output for “the” produces logits for the following token, perhaps “post”. “post” enters the cache when it is processed in the next iteration.

Timing detail: selecting a token and processing it are separate events. The Reel shows saved entries first and then a fresh pair. A standard causal attention step includes that fresh pair in the current attention calculation.

Why are the old entries still valid?

With a fixed prefix and causal masking, an earlier position cannot see future positions. Appending text therefore does not alter its earlier representation. Reuse assumes compatible model weights, token positions and attention behaviour. Editing the prefix can invalidate entries from the edit onward.

See the implementation explanation in [2].

4. The attention formula, without the mystery

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

- **QK^T:** compare queries with keys using dot products.
- **d_k:** number of components in each key/query head; scaling controls score magnitude.
- **M:** additive mask: 0 for allowed positions, $-\infty$ for blocked positions.
- **softmax:** turn scores into non-negative weights summing to 1 along the key positions.
- **Multiply by V:** form a weighted sum of value vectors.

Worked example: two-dimensional toy vectors

Suppose one head's current query is q = [1, 0]. There are two saved positions and the current position. These numbers are invented for learning; they are not extracted from a real model.

Include the current token's own key and value

Position	Key k	Value v	q · k / √2
Saved A	[1, 0]	[2, 0]	0.7071
Saved B	[0, 1]	[0, 2]	0
Current C	[1, 1]	[1, 1]	0.7071

$$\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$$
exp(scores) ≈ [2.0281, 1, 2.0281]
weights ≈ [0.4011, 0.1978, 0.4011]
output ≈ 0.4011[2,0] + 0.1978[0,2] + 0.4011[1,1]
output ≈ [1.2033, 0.7967]

The output is this head's mixed information, not the next-token probability itself. Other heads, output projections, residual connections, later layers and the language-model output head still matter.

Try it: if all three scores were equal, what would the weights be? **Answer:** one third each. The weighted output would be [1, 1].

Attention definition: [1]. Numerical example: original calculation for this guide.

5. What is saved — and what still costs work?

One attention layer, one new token

Saved K/V · Earlier positions	Fresh q, k, v · Current position	Attention result · Read combined K/V using q

A full-prefix baseline repeatedly runs the model over the growing input. Cached decoding reuses previous keys and values and processes the newly supplied token through the layers. It still runs fresh projections, attention and feed-forward computation for that token.

Count the repeated work

Take a four-token prompt and compute logits three times: at prefix lengths 4, 5 and 6. This predicts three generated tokens. The third selected token need not be processed if generation stops immediately.

Illustrative token-position processing count

Logits call	Full-prefix baseline	Cached input
1	4 positions	4 positions: prefill
2	5 positions	1 new position
3	6 positions	1 new position
Total	15	6

This is a count of token positions processed, **not a 2.5× speed benchmark**. Calls have different shapes, attention reads grow with history, and hardware behaviour matters.

Complexity in the usual full-attention case

For fixed model dimensions, the cached single-token attention calculation reads T positions, giving O(T) attention work per new token. Processing all T positions with dense attention costs $O(T^2)$ for the attention portion. Over many decoded tokens, total cached attention work still grows quadratically with generated length when the full history keeps expanding. Feed-forward work, memory traffic and kernel overhead also contribute to latency.

Takeaway: caching removes repeated prefix computation; it does not make long context free or guarantee a fixed speedup. It also does not automatically improve factual accuracy.

6. Calculate the memory bill

For a uniform, uncompressed cache with equal key and value head dimensions:

$$\text{KV bytes} = 2 \times L \times B \times T \times H_{kv} \times D \times S$$

- **2:** one key tensor plus one value tensor.
- **L:** cached attention layers.
- **B:** number of equally long sequences in this simplified batch.
- **T:** cached tokens per sequence.
- **H_{kv}:** number of key/value heads, which can differ from query heads.
- **D:** elements per key/value head.
- **S:** bytes per element; FP16/BF16 use 2 bytes.

Worked budget

Suppose $L = 32$, $B = 1$, $T = 4096$, $H_{kv} = 8$, $D = 128$ and $S = 2$.

$$2 \times 32 \times 1 \times 4096 \times 8 \times 128 \times 2$$
$$= 536{,}870{,}912 \text{ bytes}$$
$$= 512 \text{ MiB} = 0.5 \text{ GiB}$$

Doubling the cached length to 8192 doubles that payload to 1 GiB. Four equally long 4096-token sequences use 2 GiB. The model weights, other activations, allocator overhead, padding, block metadata and reserved capacity are extra.

Units: 1 MiB = 2^{20} bytes; 1 GiB = 2^{30} bytes. Decimal GB uses 10^9 bytes. The difference matters when planning GPU capacity.

Architecture changes the estimate

Multi-head attention typically has a separate K/V head per query head. Multi-query attention shares one K/V head across queries. Grouped-query attention shares K/V heads across groups. With all other toy settings fixed, changing H_{kv} from 8 to 32 makes the payload four times larger; changing it to 1 makes it eight times smaller.

The formula is a planning model, not every architecture's exact layout. Different layer types, sliding windows, compression and unequal lengths require summing their actual allocations. For unpadded variable-length sequences, replace $B \times T$ with the sum of their cached lengths.

Serving and memory background: [3]; shared K/V heads: [4], [5].

7. How engineers use this in real systems

A customer-support assistant may share a long policy prefix across requests and then generate different answers. A coding assistant may have a very long prompt but a short completion. These workloads can stress different parts of inference, so measure them separately.

Practical decisions and trade-offs

Choice	Potential benefit	What to check
Dynamic cache	Grow storage with actual use	Allocation behaviour and compilation compatibility
Static cache	Predictable capacity and shapes	Reserved space can exceed actual use
Paged allocation	Manage cache in blocks; reduce allocation waste	Block size, scheduling and sharing support
Lower-precision K/V	Reduce cache memory	Quality changes, conversion overhead and kernel support
CPU offload	Free GPU memory	Transfer latency and available bandwidth
Prefix reuse	Avoid rebuilding an eligible shared prefix	Exact token prefix and compatible computation; tenant isolation

Prefix caching reuses an existing prefix cache across eligible requests. Ordinary per-request KV caching reuses state between decoding steps. Neither is a general answer cache. Identical-looking text is not enough if token IDs, positions or the model configuration differ.

Production checklist

• Measure time to first token, per-token latency, throughput, GPU memory, active cache tokens and eviction rates using representative prompt/output lengths.

• Reserve space for weights, runtime workspace and concurrency. Enforce token limits and admission control before running out of memory.

• Keep attention masks and position indices consistent with cached length. A fast wrong answer is still wrong.

• Treat cached state as sensitive derived request data. Scope access by tenant/request and define retention and eviction rules.

• Compare cached and uncached logits within an appropriate floating-point tolerance. Separately evaluate any quantization or context truncation.

Sliding-window attention limits accessible history by model design; discarding old entries from an arbitrary full-attention model is a different approximation. PagedAttention changes memory management; it does not remove the need for valid attention state.

Implementation choices evolve; consult the current official cache strategy documentation [6].

8. A decoding loop you can reason about

This is pseudocode for a causal model. It is deliberately independent of a particular library version. The cache object contains separate state for each attention layer.

```
cache = empty_layer_caches()

logits, cache = model(prompt_ids, cache,
                      positions=prompt_positions,
                      mask=prompt_causal_mask)

for step in range(max_new_tokens):
    next_id = choose(logits[-1])
    emit(next_id)
    if next_id == EOS or step == max_new_tokens - 1:
        break
    # Feed only the token not yet processed.
    # Its mask covers saved positions AND itself.
    logits, cache = model([next_id], cache,
                          positions=[cache_length],
                          mask=all_allowed_positions)
```

Common bugs to spot

1. **Feeding the whole prompt again with a populated cache:** may duplicate positions or otherwise misuse the model interface.

2. **Wrong position index:** position-sensitive attention can give incorrect results despite plausible output.

3. **Wrong mask length:** a one-token input still needs access to all valid cached positions.

4. **Reusing state after editing text:** changed tokens can invalidate downstream entries.

5. **Sharing private state accidentally:** cache identity must include the relevant computation and isolation context.

A good verification set includes no-cache versus cached execution, multiple prompt lengths, padded batches, stopping conditions, and concurrent requests. Use deterministic settings for comparison, and account for legitimate numerical differences across kernels.

9. Glossary and misconceptions

Autoregressive generation

Predicting a next token using the preceding sequence, then repeating.

Tensor

A numerical array with one or more dimensions; a cache typically has batch, head, position and feature dimensions.

Prefill

The stage that processes the prompt and builds reusable state.

Decode

The stage that incrementally processes generated tokens.

Attention head

A learned query/key/value attention computation, combined with other heads.

Logits

Unnormalised output scores before conversion to probabilities.

Context window

The supported token span a model can use; caching does not make it unlimited.

Model weights

Learned parameters. They are distinct from a request’s changing cache.

Cache eviction

Removing stored state to reclaim memory, potentially requiring later recomputation.

Prefix cache

Reusable state for an eligible shared starting sequence.

Three things to remember

• KV caching is an inference optimisation, not training new knowledge into the model.

• It preserves intermediate numerical state, not a readable transcript or permanent personal memory.

• Full-context cache growth consumes memory even though it saves repeated computation.

10. Mini quiz — try before reading the answers

1. Which reusable vectors are normally stored: Q, K, V, or K and V?

2. Does the current query attend to its own key/value in standard causal self-attention?

3. Why can an earlier causal position’s key/value be reused after appending text?

4. What happens to the raw cache payload when T doubles with everything else fixed?

5. Is the weighted attention output itself a next-token probability distribution?

6. In the 512 MiB example, what if there are two equally long sequences?

7. Can a cache safely be reused after changing an earlier prompt token without further checks?

8. Does KV caching eliminate all work for each newly generated token?

Answers

1. **K and V.** The new query scores cached keys and combines values.

2. **Yes.** The current position is allowed; future positions are blocked.

3. **Causal masking.** Future text did not influence that earlier representation, assuming a fixed compatible prefix.

4. **It doubles.** This refers to the uniform full-history payload formula.

5. **No.** It is a mixed representation; further computation produces output logits.

6. **1024 MiB, or 1 GiB.** Excluding overhead and model weights.

7. **No.** The edit can invalidate downstream cached state.

8. **No.** The new token still needs model computation and attention to accessible history.

11. Interview practice: beginner to AI engineer

1. What is a KV cache? (Beginner)

Saved attention keys and values from processed positions, reused during incremental generation to avoid rebuilding the prefix.

2. Is it the same as model weights? (Beginner)

No. Weights are learned parameters shared across requests. KV entries depend on the processed input and usually belong to a request or an eligible shared prefix.

3. Why does the cache grow? (Beginner)

For full-history attention, each processed position adds keys and values at each cached layer. More concurrent sequences add more state too.

4. Does it make the model more knowledgeable? (Beginner)

It changes the execution strategy, not the learned knowledge. Standard exact caching aims to preserve the same computation apart from numerical effects.

5. Why are old queries generally not cached for decoding? (Intermediate)

The new position needs its own query against earlier keys and values. Earlier queries are not needed to compute the new position’s attention.

6. Explain prefill versus decode. (Intermediate)

Prefill processes the input prompt and creates the initial cache. Decode incrementally processes selected tokens, extending the cache and producing subsequent logits.

7. Why is causal masking important for reuse? (Intermediate)

An earlier token cannot attend to future tokens. Its cached state therefore remains valid as an unchanged compatible prefix is extended.

8. What are typical K/V tensor dimensions? (Intermediate)

Often [batch, KV heads, cached sequence length, head dimension] at each layer. Layouts and storage differ between runtimes, so inspect the implementation.

9. What does the memory formula exclude? (Intermediate)

Weights, other activations, temporary workspace, allocation fragmentation, metadata and reserved unused capacity. Quantized storage may also need scales or other metadata.

10. Why can decoding become memory-bandwidth limited? (AI engineer)

A single new query may read a large K/V history each step. Measure the workload: architecture, batch size, kernels and hardware determine the actual bottleneck.

11. How do MQA and GQA affect cache size? (AI engineer)

They reduce the number of KV heads relative to query heads by sharing K/V across queries. Use KV head count, not query head count, in the memory estimate.

12. What does PagedAttention solve? (AI engineer)

It enables block-based KV memory management, reducing waste and supporting sharing. It does not erase attention computation or guarantee a universal speedup.

13. How would you validate a cache implementation? (AI engineer)

Compare logits against an uncached reference under controlled settings and suitable tolerances; test position offsets, masks, padding, variable lengths, request isolation and invalidation.

14. How do you decide between quantization, offload and shorter context? (AI engineer)

First profile memory and latency. Quantization trades precision and overhead for smaller storage; offload adds transfers; truncation changes available context. Measure quality and service targets for each.

15. When is prefix reuse valid across requests? (AI engineer)

When the token prefix and the computation producing its state are compatible, including model/version, adapters, positions and masks, and the sharing policy permits it. Different continuations require separate mutable state.

16. What happens when cache capacity is exhausted? (AI engineer)

A serving system may queue, reject, evict, recompute or offload depending on policy. Admission control and token budgeting should prevent uncontrolled out-of-memory failures.

12. References and a next step

1. Attention Is All You Need — Vaswani et al., 2017 — Foundational scaled dot-product attention and causal decoder masking.
https://arxiv.org/abs/1706.03762

2. How caching works — Hugging Face Transformers — Per-layer K/V reuse, current-token attention, positions and masks.
https://huggingface.co/docs/transformers/en/cache_explanation

3. Efficient Memory Management for Large Language Model Serving with PagedAttention — Kwon et al., 2023 — Serving cache memory management, fragmentation and block allocation.
https://arxiv.org/abs/2309.06180

4. Fast Transformer Decoding: One Write-Head is All You Need — Shazeer, 2019 — Multi-query attention and the cost of reading K/V tensors.
https://arxiv.org/abs/1911.02150

5. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints — Ainslie et al., 2023 — Grouped sharing of key/value heads.
https://arxiv.org/abs/2305.13245

6. KV cache strategies — Hugging Face Transformers — Dynamic/static caches, quantization, offloading and trade-offs.
https://huggingface.co/docs/transformers/en/kv_cache

Sources reviewed 7 November 2026. Diagrams, toy vectors and arithmetic are original teaching examples, not measurements from a deployed model. This guide focuses on ordinary causal Transformer attention; other architectures can manage state differently.

Your next step: explain to a friend why a longer reply can be faster with caching yet still require more memory. Then calculate the cache payload for a configuration of your choice.

← Day 4: Speculative decoding · Learning Library